

ROAMATE

백엔드 개발 기획서 및 작업지침서

프론트엔드 1 명 + 백엔드 2 명 협업 기준

서비스 정의

한국관광공사 OpenAPI, 네이버 지도, 날씨 데이터와 생성형 AI 를 결합해
여행 중 발생한 변수에 따라 일정과 이동 경로를 재설계하는 웹 기반 AI 여행 비서

문서 범위

외부 API · REST API · 데이터 모델 · 역할 분담 · 작업 순서 · 완료 기준

문서 구성

1. 프로젝트 개요 및 범위
2. 역할 분담
3. 기술 스택 및 외부 API
4. 서버 구조 및 데이터 모델
5. 백엔드 A 작업지침 - 데이터·지도·경로
6. 백엔드 B 작업지침 - 여행·일정·AI
7. 백엔드 간 인터페이스
8. 공통 API 규칙
9. 개발 일정
10. 최종 완료 기준

문서 사용 방법

백엔드 담당자 A와 B는 자신의 담당 절을 기준으로 개발하고, 7장 인터페이스와 8장 공통 규칙은 반드시 동일하게 적용합니다.

1. 프로젝트 개요 및 범위

1.1 핵심 사용자 흐름

여행 일정 조회

- 현재 위치·사용자 상태 입력
- 날씨·운영시간·이동시간 분석
- 일정 영향 여부 판단
- 대체 관광지 검색
- AI 변경 일정 생성
- 변경 전후 비교
- 사용자 승인
- 일정 적용 및 저장

1.2 필수 구현 기능

- 관광지 검색·상세·주변 조회
- 여행 및 일정 CRUD
- 네이버 자동차 경로 계산
- 현재 위치 기반 주소 조회
- 현재 날씨 및 위험 요소 조회
- 사용자 상태 저장
- 일정 영향 분석
- 대체 관광지 후보 생성
- Spring AI 일정 재설계
- 변경 전후 비교
- 변경안 적용·거절

- 변경 이력 저장

1.3 제외 기능

제외 기능	처리 방향
대중교통 경로 직접 계산	MVP 제외
도보 내비게이션	MVP 제외
실시간 식당 웨이팅	고도화
관광지 실시간 혼잡도	고도화
예약·결제	고도화
음성 안내	고도화

2. 역할 분담

담당	핵심 책임	브랜치
프론트엔드	네이버 지도 UI, 일정 타임라인, AI 바텀시트, API 연동	feature/frontend
백엔드 A	외부 데이터, 장소, 경로, 주소, 날씨	feature/backend-data-map
백엔드 B	여행, 일정, Context, Impact, Spring AI, Replan	feature/backend-trip-ai

분업 원칙

백엔드 A 는 외부 API 와 정규화된 데이터 제공까지 담당합니다. 백엔드 B 는 여행 도메인과 AI 로직을 담당합니다. Controller 간 직접 호출은 금지합니다.

3. 기술 스택 및 외부 API

3.1 기술 스택

구분	기술
언어·프레임워크	Java 21, Spring Boot
웹	Spring Web, Spring Validation
DB	MySQL, Spring Data JPA
AI	Spring AI
외부 호출	RestClient
기타	Lombok, Postman, DBeaver, GitHub

3.2 외부 API

API	사용 목적	핵심 출력
한국관광공사 OpenAPI	관광지 검색·상세·이미지·운영정보	contentId, 위치, 이미지, 운영시간
NAVER Maps Dynamic Map	프론트 지도 표시	지도, 마커, Polyline
NAVER Directions	자동차 경로 계산	거리, 소요시간, 경로 좌표
NAVER Reverse Geocoding	좌표를 주소로 변환	시·구·도로명 주소
날씨 API	현재 날씨와 위험요소	기온, 강수, 풍속, 위험 여부
Spring AI	일정 재설계와 설명	구조화된 변경 일정

3.3 Spring AI 제한 규칙

- 후보 목록에 없는 장소 생성 금지
- contentId 임의 생성·변경 금지
- 이동시간 임의 생성 금지
- 날씨·운영시간 추측 금지
- 사용자 승인 전 일정 적용 금지

4. 서버 구조 및 데이터 모델

4.1 패키지 구조

com.roamate	
├─ common	exception, response
├─ config	
├─ place	controller, service, dto, entity
├─ route	controller, service, dto
├─ weather	
├─ location	
├─ trip	controller, service, repository, dto, entity
├─ schedule	
├─ context	
├─ impact	
├─ replan	

assistant

externalkto, naver, weather

4.2 데이터 모델

테이블	핵심 필드	용도
User	id, email, nickname	사용자
UserPreference	walkingLevel, preferIndoor, theme	여행 선호
Trip	title, regionCode, startDate, status	여행 단위
Place	contentId, title, lat/lng, openingHours	관광지 캐시
Schedule	order, start/end, travelMinutes, mustVisit	일정 항목
TravelContext	위치, 상태, 날씨	현재 여행 상황
Replan	contextId, summary, status	변경안
ReplanItem	actionType, order, time, reason	변경 상세

5. 백엔드 A 작업지침 - 데이터·지도·경로

5.1 담당 범위

- 한국관광공사 OpenAPI 연동
- 네이버 Directions 연동
- Reverse Geocoding 연동
- 날씨 API 연동
- 관광지 데이터 정규화
- 후보 장소 검색·점수 계산
- 외부 API 예외 처리

5.2 관광지 API

Method	Endpoint	기능
GET	/api/places	키워드·지역 관광지 검색
GET	/api/places/nearby	현재 위치 주변 검색
GET	/api/places/{contentId}	상세정보
GET	/api/places/{contentId}/images	이미지 목록

5.3 관광지 응답 규격

```
{
  "contentId": "126848",
  "contentType": "14",
  "title": "부산현대미술관",
  "address": "부산광역시 사하구 낙동남로 1191",
  "latitude": 35.1089,
  "longitude": 128.9424,
  "imageUrl": "https://...",
  "indoor": true,
  "openingHours": "10:00~18:00",
  "closedDay": "월요일",
  "source": "KT0"
}
```

5.4 데이터 변환 규칙

- mapX 는 longitude, mapY 는 latitude 로 변환
- 좌표가 없는 장소는 제외
- 이미지가 없으면 null
- 운영시간이 없으면 "정보 없음"
- 외부 응답을 PlaceResponse 로 변환 후 반환

5.5 자동차 경로 API

Method	Endpoint	검증
POST	/api/routes/driving	출발·도착 필수, 경유지 최대 5 개, 좌표 범위 검증

요청

```
{
  "start": {"latitude":35.1796,"longitude":129.0756},
  "goal": {"latitude":35.1532,"longitude":129.1187},
  "waypoints": [{"latitude":35.1661,"longitude":129.1012}],
  "option": "trafast"
}
```

응답

```
{
  "distanceMeters": 12840,
  "durationMillis": 1860000,
  "durationMinutes": 31,
  "path": [{"longitude":129.0756,"latitude":35.1796}]
}
```

5.6 주소·날씨 API

Method	Endpoint	응답 핵심
GET	/api/locations/reverse-geocode	city, district, roadAddress
GET	/api/weather/current	condition, temperature, rainProbability, windSpeed, outdoorRisk

5.7 후보 장소 점수

항목	점수
기본 점수	100
이동시간 1 분당	-1.5
실내 장소	+20
현재 운영 중	+25
사용자 선호 일치	+15
기존 일정에 없는 장소	+10
정보 완성도 높음	+10
운영 종료	-100

완료 기준

Postman 에서 관광지 검색, 주변 검색, 경로, 주소, 날씨 API 가 모두 정상 응답해야 합니다. 외부 API 실패 시 502 오류를 반환하고 키는 로그에 남기지 않습니다.

6. 백엔드 B 작업지침 - 여행·일정·AI

6.1 담당 범위

- 여행 및 일정 CRUD
- 현재 여행 상태 저장
- 일정 영향 분석
- Spring AI 일정 재설계
- AI 결과 검증
- 변경안 저장·적용·거절
- 변경 이력 관리

6.2 여행·일정 API

Method	Endpoint	기능
POST	/api/trips	여행 생성
GET	/api/trips/{tripId}	여행 조회
PATCH	/api/trips/{tripId}	여행 수정
DELETE	/api/trips/{tripId}	여행 삭제
POST	/api/trips/{tripId}/schedules	일정 추가
PATCH	/api/trips/{tripId}/schedules/{scheduleId}	일정 수정
DELETE	/api/trips/{tripId}/schedules/{scheduleId}	일정 삭제
PATCH	/api/trips/{tripId}/schedules/order	순서 변경

6.3 여행 상태 API

```
POST /api/trips/{tripId}/context
{
  "latitude": 35.1532,
  "longitude": 129.1187,
  "currentTime": "2026-07-20T14:20:00",
  "currentScheduleId": 3,
  "userCondition": "TIRED"
}
```

응답

```
{
  "contextId": 41,
  "recordedAt": "2026-07-20T14:20:00"
}
```

사용자 상태: NORMAL, TIRED, HUNGRY, MINIMIZE_WALKING, PREFER_INDOOR

6.4 일정 영향 분석

조건	Severity
비 + 야외 일정	HIGH
강풍 + 해안 일정	HIGH

조건	Severity
도착 예상 시각 > 마감시간	HIGH
TIRED	MEDIUM
HUNGRY + 식사 일정 없음	MEDIUM
변수 없음	LOW

6.5 일정 재설계 API

```
POST /api/trips/{tripId}/replans
{
  "contextId": 41,
  "options": {
    "preferIndoor": true,
    "minimizeWalking": true,
    "preserveMustVisit": true,
    "maximumPlaces": 4
  }
}
```

6.6 내부 처리 순서

1. 현재 여행과 남은 일정 조회
2. 영향 일정 추출
3. 백엔드 A 의 주변 장소 검색 호출
4. 후보별 이동시간 계산
5. 점수 상위 10 개 선정
6. Spring AI 호출
7. AI 결과 검증
8. Replan 및 ReplanItem 저장
9. 변경 전후 비교 반환

6.7 AI 입력·제약·검증

구분	내용
입력	현재 시각·위치·날씨·사용자 상태·남은 일정·필수 장소·후보·이동시간·점수
제약	후보 외 장소 금지, contentId 변경 금지, 운영 종료 금지, 시간 충돌 금지, 최대 장소 수 준수
검증	허용 contentId, 필수 장소 유지, 시간 중복, 운영시간, 현재 시각 이후, 이동 시간 반영

6.8 AI 실패 처리

1차 검증 실패 → AI 재호출 1회
 2차 검증 실패 → 규칙 기반 대체 일정 반환
 검증 실패 결과 → DB 저장 금지

6.9 변경안 적용·거절

Method	Endpoint	처리
POST	/api/trips/{tripId}/replans/{replanId}/apply	기존 일정 백업, 제안 일정 적용, 상태 APPLIED
POST	/api/trips/{tripId}/replans/{replanId}/reject	기존 일정 유지, 상태 REJECTED

7. 백엔드 간 인터페이스

백엔드 B는 백엔드 A의 Controller를 호출하지 않고 Service 인터페이스를 의존성 주입받아 사용합니다.

```
public interface PlaceSearchService {
    List<PlaceCandidate> searchNearby(
        double latitude,
        double longitude,
        CandidateSearchCondition condition
    );
}

public interface RouteCalculationService {
    RouteResult calculateDrivingRoute(
        Coordinate start,
        Coordinate goal,
        List<Coordinate> waypoints
    );
}

public interface WeatherQueryService {
    WeatherSnapshot getCurrentWeather(
        double latitude,
        double longitude
    );
}
```

호출 규칙

외부 API 호출 구현은 백엔드 A가 담당하고, 백엔드 B는 위 인터페이스만 사용합니다. 인터페이스 DTO는 공동 합의 후 변경합니다.

8. 공통 API 규칙

8.1 오류 응답

```
{
  "code": "PLACE_NOT_FOUND",
  "message": "관광지를 찾을 수 없습니다.",
  "timestamp": "2026-07-20T14:20:00"
}
```

8.2 HTTP 상태코드

상태	의미
200	조회·수정 성공
201	생성 성공

상태	의미
204	삭제 성공
400	요청값 오류
404	데이터 없음
409	현재 상태에서 처리 불가
502	외부 API 오류
500	서버 내부 오류

8.3 공통 개발 규칙

- API 키는 환경변수로 관리
- Client Secret 과 서비스키는 프론트에 노출 금지
- 외부 API 응답을 그대로 반환하지 않음
- AI 결과를 검증한 뒤 저장
- 모든 날짜·시간은 ISO-8601 형식
- Controller 는 요청·응답만 담당
- 비즈니스 로직은 Service 에 작성

9. 개발 일정

일차	백엔드 A	백엔드 B	통합 목표
1 일	API 키·외부 API 테스트	프로젝트·DB·엔티티	Health, 샘플 Trip
2 일	관광지 검색·상세·주변	여행·일정 CRUD	지도 마커·일정 출력
3 일	Directions·주소·날씨	Context·Impact	위치 기반 상황 분석
4 일	후보 점수·외부 예외	Spring AI·검증	Replan JSON 반환
5 일	경로 비교·통합 지원	적용·거절·이력	프론트 일정 갱신
6 일	Postman·API 문서	테스트·예외 처리	통합 테스트
7 일	프론트 통합 수정	프론트 통합 수정	발표 시나리오 완료

10. 최종 완료 기준

10. 여행 조회
11. 현재 위치·사용자 상태 저장
12. 현재 날씨 조회
13. 기존 일정 영향 분석
14. 주변 대체 관광지 검색
15. 후보별 자동차 이동시간 계산
16. AI 변경 일정 생성
17. 변경 전후 비교 반환
18. 사용자 승인
19. 일정 DB 교체
20. 최신 일정 반환

완료 판정

위 흐름이 Postman 과 프론트 화면에서 모두 정상 동작하면 백엔드 개발 완료로 판정합니다.

부록. 담당자별 제출 체크리스트

백엔드 A	백엔드 B
관광공사 Client Directions Client Reverse Geocoding Client 날씨 Client Place·Route·Weather API 후보 검색·점수 로직 Postman Collection	여행·일정 엔티티 Trip·Schedule CRUD TravelContext·Impact API Spring AI 서비스 AI 검증 로직 Replan 적용·거절 DB ERD